



综合实训报告

转转校园二手物品交易平台设计与实现

姓 名：张可凡

学 号：2024307020XXX

班 级：计科 210X 班

指导老师：徐士伟

中国武汉

二〇二六年七月

2026.07

目录

1	实训目的及内容	3
1.1	实训目的	3
1.2	实训内容	3
1.3	实训安排	3
1.3.1	团队组成	3
1.3.2	开发计划	3
2	需求分析	4
2.1	功能需求	4
2.1.1	用户管理模块	4
2.1.2	商品管理模块	5
2.1.3	订单交易模块	5
2.1.4	消息与通知模块	5
2.1.5	后台管理模块	5
2.2	非功能需求	5
3	系统设计	6
3.1	体系结构设计	6
3.1.1	前端架构	6
3.1.2	后端架构	6
3.1.3	部署架构	7
3.2	数据设计	7
3.2.1	核心数据库表	7
4	系统实现	8
4.1	JWT 双 Token 认证机制	8
4.2	订单状态流转引擎	8
4.3	商品搜索与筛选	9
4.4	消息即时通信	9
4.5	数据统计与可视化	9
5	系统测试	9
5.1	测试策略	9

5.2	测试用例设计	10
5.3	测试结果	10
5.3.1	单元测试	10
5.3.2	接口测试	11
5.3.3	性能优化	11
6	结论与体会	11
6.1	结论	11
6.2	个人体会	12
7	参考文献	12

1 实训目的及内容

1.1 实训目的

本次综合实训以软件工程理论为指导，通过开发一个完整的校园二手物品交易平台，达到以下目的：

1. 巩固和深化软件工程、Web 前端开发技术、Java 程序设计、数据库原理与应用、计算机网络等课程的理论知识。
2. 掌握前后端分离架构的开发模式，熟悉 SpringBoot + Vue3 主流技术栈的工程化实践。
3. 培养团队协作能力与项目管理能力，体验从需求分析、系统设计、编码实现到测试部署的完整软件生命周期。
4. 提升独立解决实际问题的能力，包括技术选型、架构设计、性能优化、安全防护等综合工程素养。

1.2 实训内容

本项目”转转”是一个面向高校学生的 C2C 二手物品交易 Web 应用系统，旨在解决校园闲置物品循环利用的需求。系统采用前后端分离架构，后端基于 Spring Boot 3 + Spring Security + JWT 实现 RESTful API，前端基于 Vue 3 + TypeScript + Element Plus 构建用户界面，使用 MySQL 存储业务数据，Redis 提供缓存加速。

系统实现了完整的交易闭环，包括用户管理、商品管理、购物车与订单交易、站内消息沟通、后台管理五大核心模块，并提供 Docker Compose 一键部署方案。

1.3 实训安排

1.3.1 团队组成

实训小组共 5 人，分工如下：

1.3.2 开发计划

项目周期为 2026 年 7 月 4 日至 2026 年 7 月 10 日，共分四个阶段：

表 1: 团队成员与角色分工

成员	角色	职责
张可凡（队长）	全栈/项目管理	Docker 部署、后台前端、Postman 测试、文档
阮崇轩	后端开发 A	项目骨架、用户模块、商品模块、安全框架
刘轩霆	后端开发 B	购物车、订单模块、消息模块、后台 API
胡欣悦	前端开发 C	首页、登录注册、个人中心、收藏、通知
李硕	前端开发 D	商品页面、购物车、订单页面、管理后台

- **第 1 天（7 月 4 日）：**环境搭建与基础框架——Git 仓库创建、Docker 编排、前后端项目脚手架、数据库表设计。
- **第 2–3 天（7 月 5 日–6 日）：**核心功能开发（用户 + 商品）——安全框架集成、注册登录、商品 CRUD、分类与收藏。
- **第 4–5 天（7 月 7 日–8 日）：**核心功能开发（订单 + 消息）——购物车、订单状态流转、站内信、系统通知。
- **第 6–7 天（7 月 9 日–10 日）：**管理后台 + 联调测试——用户/商品/订单管理、数据统计、全量接口联调、文档编写。

2 需求分析

转转校园二手交易平台作为面向高校学生的 C2C 交易平台，主要服务三类用户角色：游客（未登录用户）、普通用户（买家和卖家双重身份）以及管理员。系统的顶层用例如图1所示。

图 1: 系统顶层用例图

2.1 功能需求

系统包含以下五大核心功能模块：

2.1.1 用户管理模块

支持用户的完整生命周期管理。游客可通过邮箱/手机号验证码注册成为平台用户，注册后可使用用户名或邮箱或手机号登录系统。系统采用 JWT 双 Token 机制（Access Token 2 小时过期，Refresh Token 7 天过期）实现无状态认证，支持密码找回、Token 无感刷新、安全退出（Token 加入 Redis 黑名单）等功能。用户可在个人中心编辑昵称、简介、性别，上传头像，管理多个收货地址并设置默认地址。

2.1.2 商品管理模块

支持商品的完整生命周期管理。卖家可发布商品，填写标题、描述（富文本）、价格、原价、成色，选择分类并上传多张图片。商品发布后需管理员审核方可上架展示。买家可通过首页推荐、分类导航、关键词搜索（全文索引）浏览商品，支持按分类、价格区间、成色筛选，按价格、时间、热度排序。买家可收藏感兴趣的商品，卖家可查看和管理自己发布的商品。

2.1.3 订单交易模块

实现完整的交易闭环。买家可将商品加入购物车统一结算，下单时选择收货地址。订单状态按“待付款 → 待发货 → 待收货 → 已完成”流转，系统提供模拟支付功能，卖家可录入物流信息发货，买家确认收货后可对交易进行评分和评价。所有状态变更均记录订单日志，便于追踪溯源。

2.1.4 消息与通知模块

支持买卖双方的即时沟通。买家可通过商品详情页的“联系卖家”发起会话，双方可互发文本消息或图片消息，支持会话列表查看、聊天记录分页查询、消息已读/未读标记和未读计数。系统自动推送订单状态变更、审核结果等系统通知。

2.1.5 后台管理模块

为管理员提供全面的管理功能。包括用户管理（用户列表、搜索筛选、禁用/启用、修改角色）、商品审核（待审核列表、审核通过/驳回、违规强制下架）、订单管理（按状态/单号/日期查询、异常状态处理）、数据统计（注册用户量、商品发布量、成交量、分类占比饼图、趋势折线图，基于 ECharts 实现）以及公告管理（发布/编辑/删除系统公告）。

2.2 非功能需求

- **安全性：**密码 BCrypt 加密存储，JWT 无状态认证，接口 RBAC 权限控制，SQL 注入防护（参数白名单校验），XSS 防护。
- **性能：**Redis 缓存热点商品数据，数据库复合索引优化，N+1 查询问题修复，前端 Gzip 压缩与路由懒加载。
- **可用性：**响应式 UI 设计，统一异常处理与友好的错误提示，Token 自动刷新机制保证会话连续性。

-
- **可维护性：**前后端分离架构，RESTful API 规范，统一返回结果封装，完善的订单日志审计。

3 系统设计

3.1 体系结构设计

系统采用前后端分离的 B/S 架构，前端与后端通过 RESTful API 进行数据交互。整体架构分为四层，如图2所示。

图 2: 系统体系结构图

3.1.1 前端架构

前端基于 Vue 3 + TypeScript + Vite 构建，采用组合式 API 开发模式。使用 Element Plus 提供 UI 组件，Pinia 管理全局状态，Vue Router 实现路由控制。Axios 封装请求拦截器自动携带 JWT Token，响应拦截器统一处理错误并支持 Token 自动刷新。前端构建通过 Vite 插件实现 Element Plus 按需导入、Gzip 压缩和代码分割。

3.1.2 后端架构

后端采用 Spring Boot 3 + MyBatis-Plus + Spring Security 技术栈，遵循经典的三层架构设计：

- **Controller 层：**接收 HTTP 请求，参数校验，调用 Service 层处理业务，返回统一封装的 Result 对象。
- **Service 层：**业务逻辑处理，包括用户认证、商品管理、订单流转、消息通信等核心业务。
- **DAO 层：**基于 MyBatis-Plus 操作数据库，支持 Lambda 查询和分页插件。

安全方面，Spring Security 配置为无状态会话模式，通过 JwtAuthenticationFilter 拦截请求，从请求头提取 Token 解析用户信息并设置 SecurityContext。Redis 用于缓存验证码、黑名单 Token、热点商品数据等。

3.1.3 部署架构

系统采用 Docker Compose 编排四容器架构：Nginx（前端静态资源服务与反向代理）、Spring Boot 后端、MySQL 8.0 数据库、Redis 7 缓存。所有服务通过内部网络通信，MySQL 和 Redis 不对外暴露端口，后端仅通过 Nginx 反向代理对外暴露。

3.2 数据设计

系统共设计 13 张核心业务表，涵盖用户、商品、订单、消息等全部业务实体。表结构和关系如图3所示。

图 3: 数据库 E-R 图

3.2.1 核心数据库表

表 2: 核心数据库表清单

表名	用途	关键字段
user	用户表	username, password_hash, email, phone, role
category	商品分类表	name, parent_id, level
product	商品表	user_id, category_id, title, price, status
product_image	商品图片表	product_id, url, sort_order
address	收货地址表	user_id, receiver, phone, province, city
order	订单表	order_no, buyer_id, seller_id, status
order_log	订单日志表	order_id, from_status, to_status
cart	购物车表	user_id, product_id, quantity
message	消息表	from_user_id, to_user_id, content, type
favorite	收藏表	user_id, product_id
review	评价表	order_id, from_user_id, rating
notification	通知表	user_id, title, content, type
announcement	公告表	admin_id, title, content, status

数据表设计特点：所有表使用 InnoDB 引擎和 utf8mb4 字符集；采用软删除策略（deleted_at 字段配合 MyBatis-Plus @TableLogic）；created_at 和 updated_at 通过 MyMetaObjectHandler 自动填充；product 表含全文索引支持商品搜索；通过复合索引优化消息会话查询、未读计数、收藏排序等高频查询场景。

4 系统实现

4.1 JWT 双 Token 认证机制

系统采用 Access Token + Refresh Token 双 Token 机制实现无状态认证。Access Token 有效期为 2 小时，存储用户 ID 和角色信息；Refresh Token 有效期为 7 天，用于无感刷新 Token。

Listing 1: JWT 认证过滤器核心逻辑

```
@Override
protected void doFilterInternal(HttpServletRequest request,
    HttpServletResponse response, FilterChain chain) {
    String authHeader = request.getHeader("Authorization");
    if (authHeader != null && authHeader.startsWith("Bearer_")) {
        String token = authHeader.substring(7);
        // check blacklist
        if (redisTemplate.hasKey("blacklist:token:" +
            jwtUtil.getId(token))) {
            chain.doFilter(request, response);
            return;
        }
        // parse token and set authentication
        Claims claims = jwtUtil.parseToken(token);
        SecurityContextHolder.getContext().setAuthentication(
            new UsernamePasswordAuthenticationToken(
                claims.getSubject(), null, authorities));
    }
    chain.doFilter(request, response);
}
```

4.2 订单状态流转引擎

订单模块是整个系统的核心业务，涉及多个状态的流转和数据一致性。采用状态模式设计，每个状态变更均记录日志，关键操作（创建订单扣减库存）通过 @Transactional 保证原子性。

图 4: 订单状态流转图

订单状态流转路径为:

待付款 $\xrightarrow{\text{支付}}$ 待发货 $\xrightarrow{\text{发货}}$ 待收货 $\xrightarrow{\text{收货}}$ 已完成 $\xrightarrow{\text{评价}}$ 已评价

4.3 商品搜索与筛选

商品模块支持多条件组合查询, 包括分类筛选、价格区间、成色筛选、关键词搜索以及按价格/时间/热度排序。基于 MyBatis-Plus 的 LambdaQueryWrapper 动态拼接查询条件, 配合 MySQL 全文索引实现关键词搜索。为防范 SQL 注入, 排序字段采用白名单机制, 仅允许预定义字段参与排序。

4.4 消息即时通信

站内信模块实现了买家与卖家的实时沟通功能。消息表记录所有通信内容, 按会话 (Conversation) 维度组织, 通过 from_user_id 和 to_user_id 的组合查询生成会话列表。系统支持消息已读/未读标记, 并通过 Redis 缓存未读计数提升查询性能。

4.5 数据统计与可视化

管理后台的数据统计模块基于 ECharts 实现数据可视化, 包括用户增长趋势折线图、商品分类分布饼图、每日订单成交量柱状图等。后端通过 SQL 聚合查询获取统计数据, 前端使用 ECharts 图表组件渲染展示。

5 系统测试

5.1 测试策略

系统采用多层次测试策略: 后端使用 JUnit 5 + Mockito 进行单元测试和集成测试, H2 内存数据库作为测试环境; 前端使用 Vitest + Vue Test Utils 进行组件和 Store 测试; 全量 API 通过 Postman 集合进行接口验证。

5.2 测试用例设计

以订单模块的核心功能为例，采用黑盒测试法设计的测试用例如表3所示。

表 3: 订单模块核心测试用例

测试用例编号	ORDER_CORE_001
测试项目	订单创建与支付流程
测试标题	正常创建订单并完成支付
重要级别	高
预置条件	买家已登录，商品在售且库存充足，买家有默认收货地址
输入	商品 ID、数量 1、收货地址 ID
操作步骤	调用创建订单接口； 调用支付接口
预期输出	订单创建成功，状态为”待付款”；支付后状态变更为”待发货”

表 4: 商品搜索测试用例

测试用例编号	PRODUCT_SEARCH_001
测试项目	商品多条件组合搜索
测试标题	按分类 + 价格区间 + 排序搜索商品
重要级别	中
预置条件	数据库中已存在多个分类的商品数据
输入	categoryId=1, minPrice=100, maxPrice=500, sortBy=price, order=asc
操作步骤	调用商品列表接口，传入上述参数
预期输出	返回分类 ID 为 1、价格在 100-500 元之间的商品列表，按价格升序排列

5.3 测试结果

5.3.1 单元测试

后端共完成 37 个单元测试，覆盖 Result 工具类、JwtUtil 工具类、UserServiceImpl、AddressServiceImpl 以及 Controller 集成测试；前端共完成 13 个单元

测试，覆盖 auth 工具函数、组件渲染、Result 数据格式以及 Pinia Store。全部 50 个测试均通过，成功率 100%。

表 5: 单元测试统计

测试范围	测试数	通过	通过率
后端单元测试	37	37	100%
前端单元测试	13	13	100%
总计	50	50	100%

5.3.2 接口测试

通过 Postman 集合对全部 60+ 个 API 端点进行验证，涵盖首页（3 个）、用户认证（6 个）、用户信息（3 个）、地址管理（5 个）、商品（11 个）、购物车（4 个）、订单（6 个）、消息（5 个）、管理后台（8 个）、上传（1 个）十大模块，全部接口通过验证。

5.3.3 性能优化

在测试过程中发现并修复了以下关键问题：

1. **N+1 查询问题：**商品列表每页加载 41 条 SQL 语句，通过批量加载分类和卖家信息优化至 3 条 SQL，性能提升约 13 倍。
2. **SQL 注入漏洞：**‘ProductMapper.xml’中排序字段直接拼接参数，改为白名单校验机制。
3. **数据库索引缺失：**新增 4 个复合索引优化消息会话查询、未读计数、收藏排序和卖家商品筛选。
4. **逻辑错误：**‘countProductByUser()’查错表，修正为正确的 Mapper 方法。

6 结论与体会

6.1 结论

经过为期一周的集中开发与测试，“转转”校园二手物品交易平台已成功实现全部预设功能。系统基于 Spring Boot 3 + Vue 3 前后端分离架构，实现了用户管理、商品管理、购物车与订单交易、站内消息通信、后台管理五大核心模块，

共计 60+ 个 RESTful API 和 15 个前端页面。系统通过了 50 个单元测试和全量 Postman 接口测试，成功率为 100%，并已通过 Docker Compose 实现一键部署。

项目的核心成果包括：实现了基于 JWT 双 Token 机制的安全认证体系，保障了接口访问安全；设计了完整的订单状态流转引擎，支持从下单到评价的全流程闭环；构建了基于 Redis 的缓存加速机制，提升了系统响应性能；同时修复了 N+1 查询、SQL 注入等关键质量和安全问题。

6.2 个人体会

通过本次综合实训，我在以下几个方面获得了显著提升：

系统分析与设计能力：在需求分析阶段，学习如何将模糊的业务需求转化为明确的系统功能和数据结构；在架构设计阶段，深入理解了前后端分离架构的优势，掌握了 RESTful API 的设计规范和数据库表设计的范式理论。

技术实践能力：通过实际开发，巩固了 Spring Boot 框架的核心特性（IoC、AOP、事务管理），熟练掌握了 MyBatis-Plus 的 Lambda 查询和分页插件使用，理解了 JWT 无状态认证的工作原理和实现方式。前端方面，深入体验了 Vue 3 组合式 API 的开发模式，掌握了 Pinia 状态管理和 Axios 拦截器等关键技术。

项目管理与团队协作：作为队长，负责全组任务分配、进度协调和最终质量把控。使用 Git 进行版本控制和分支管理，确保多人协作的代码一致性。每日召开晨会同步进度，及时解决联调中遇到的技术问题。

问题解决能力：在开发过程中遇到了一系列技术挑战——Docker 容器启动顺序控制、订单库存扣减的原子性保证、前后端字段命名不一致等。通过查阅官方文档和技术社区，逐一找到了合理的解决方案，积累了宝贵的实战经验。

质量意识：测试环节让我深刻认识到软件质量的重要性。通过多层次测试策略，发现了 N+1 查询性能瓶颈和 SQL 注入安全漏洞等容易被忽视的问题，意识到好的软件不仅要“能用”，更要“好用”、“安全”。

7 参考文献

1. 尤雨溪. Vue.js 3 设计与实现 [M]. 北京: 人民邮电出版社, 2023.
2. Walls C. SpringBoot 实战 [M]. 第 4 版. 北京: 人民邮电出版社, 2022.
3. Widenius M. MySQL 必知必会 [M]. 北京: 人民邮电出版社, 2020.
4. 黄健宏. Redis 设计与实现 [M]. 北京: 机械工业出版社, 2014.
5. Richardson L, Ruby S. RESTful Web APIs[M]. O'Reilly Media, 2013.

-
6. 王松. Spring Boot 整合开发实战 [M]. 北京: 清华大学出版社, 2023.
 7. 张卫朋. Vue.js 3 从入门到精通 [M]. 北京: 人民邮电出版社, 2024.